# 1.What are CICD pipelines

CI/CD stands for Continuous Integration and Continuous Deployment. It is an automated process that runs every time a developer pushes code to the repository. The pipeline builds the code, runs tests, and deploys it to the target environment without manual intervention.

### CI — Continuous Integration

- Automatically triggered on every code push or pull request
- Compiles and builds the application
- Runs unit tests and integration tests
- Performs code quality and security scans
- Notifies the developer if any step fails

### CD — Continuous Deployment

- Deploys the build artifact to the target environment automatically
- Runs post-deployment smoke tests to verify the release
- Triggers rollback if deployment health checks fail

### Pipeline Stages

1. Code Commit — Developer pushes code to the repository
2. Build — Code is compiled and packaged into a deployable artifact
3. Unit Test — Individual components are tested in isolation
4. Integration Test — Multiple components are tested together
5. Code Analysis — Security scanning and code quality checks
6. Deploy to Staging — Artifact is deployed to the staging environment
7. Smoke Test — Basic end-to-end checks run against staging
8. Deploy to Production — Artifact is promoted to the live environment

**Common Tools** — GitHub Actions, GitLab CI/CD, Jenkins, CircleCI, Azure DevOps

## 2.How do Application/product go Live?

"Going Live" is the final step of the release process. While every company has a slightly different flavor, the standard path looks like this:

1. **Code Commit** — Developers finish a feature and merge it into the main codebase.
2. **The Build** — The CI tool compiles the code and creates an artifact (like a Docker image or a .jar file).
3. **Automated Testing** — The artifact is put through a battery of tests (integration, security, and performance).
4. **Staging / UAT** — The app is deployed to a Pre-Live environment for final human approval.
5. **Production Deployment** — The artifact is pushed to the live servers. This often uses strategies like Blue-Green Deployment (switching traffic from an old version to a new one) to ensure zero downtime — meaning users experience no interruption or outage during the deployment. If the new version has issues, traffic is immediately switched back to the old version as a rollback.

## 3.What Are Different Environments While Working On A Project?

A software project uses multiple separate environments to safely develop, test, and release the application. Each environment serves a specific purpose and has a defined audience.

**Local** — Developer writes and tests code on their own machine before sharing with the team.

**Development** — All developers integrate and combine their code into a shared server. This environment is frequently updated and may be unstable at times.

**QA / Testing** — A dedicated environment where the testing team verifies functionality, runs test cases, and reports bugs before the code moves further.

**Staging** — A production-identical environment used for final validation. Product managers and stakeholders review and approve the release here.

**Production** — The live environment accessed by real end users. Highest stability is required and all changes go through a formal release process.

Code always moves in sequence: Local → Development → QA → Staging → Production. Each environment acts as a gate, ensuring only stable and tested code advances to the next stage.